

Día 04 · Tema 2 — Convolución, pooling y transfer learning

1. Por qué falla un MLP sobre imágenes

Una imagen $H \times W \times C$ aplanada tiene HWC entradas. Un MLP con anchura D en la primera capa requiere $W_1 \in \mathbb{R}^{D \times HWC}$, es decir $D \cdot H \cdot W \cdot C$ pesos sólo en la primera capa: para $224 \times 224 \times 3$ y $D=1024$ son $1,5 \cdot 10^8$ parámetros. Peor todavía, el MLP es *sensible a traslaciones*: trasladar un píxel a la derecha cambia todos los pesos relevantes. Las dos *inductive biases* que motivan la CNN son: **(i)** localidad (píxeles cercanos son más informativos entre sí) y **(ii)** equivarianza por traslación (un objeto es el mismo objeto en cualquier posición).

2. La operación de convolución

Dado un *feature map* $X \in \mathbb{R}^{H \times W \times C_{in}}$ y un *kernel* $K \in \mathbb{R}^{k \times k \times C_{in} \times C_{out}}$, la salida $Y \in \mathbb{R}^{H' \times W' \times C_{out}}$ se define como

$$Y_{i,j,o} = \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} \sum_{c=1}^{C_{in}} X_{i+u, j+v, c} K_{u,v,c,o} + b_o. \quad (1)$$

$X_{i,j,c}$ intensidad del píxel (i, j) en el canal de entrada c
 $K_{u,v,c,o}$ peso del kernel en la posición (u, v) , conectando $c \rightarrow o$
 b_o sesgo del canal de salida o
 k tamaño del kernel (típicamente 3, 5, 7)
 C_{in}, C_{out} canales de entrada y salida

Pesos compartidos: el mismo K se aplica en todas las posiciones, con coste $k^2 C_{in} C_{out}$ pesos por capa, *independiente* de la resolución espacial.

2.1. Dimensiones de salida con padding y stride

Con *stride* s y *padding* p , la altura de salida es $H' = \lfloor (H + 2p - k)/s \rfloor + 1$ (igual para W).

3. Pooling

Reducir resolución espacial gana invariancia a pequeñas traslaciones y reduce cómputo. Max-pooling sobre una ventana $k_p \times k_p$ es

$$Y_{i,j,c} = \max_{0 \leq u, v < k_p} X_{i \cdot s + u, j \cdot s + v, c}. \quad (2)$$

Average pooling sustituye $\max$ por $\frac{1}{k_p^2} \sum$. En arquitecturas modernas se prefiere stride en la convolución a max-pool agresivo.

4. Arquitecturas canónicas y residual

La **ResNet** introdujo el bloque residual:

$$y = \mathcal{F}(x; \{W_i\}) + x, \quad (3)$$

$\mathcal{F}$ una o dos capas conv + activación
 $+x$ la *skip connection* que sortea $\mathcal{F}$

La skip resuelve el problema del gradiente que se desvanece en redes profundas; permite entrenar redes de 50, 101, 152 capas sin colapsar.

5. Transfer learning

Sea un modelo f_{θ_0} preentrenado en $\mathcal{D}_{src}$ (ImageNet, COCO, LAION). En *transfer* dividimos $\theta = (\theta_b, \theta_h)$ en *backbone* y *head*; sólo entrenamos θ_h (o ambos con distinto η). La pérdida en el dataset objetivo $\mathcal{D}_{tgt}$ de tamaño n es la habitual pero θ_0 ya está cerca del óptimo:

$$\hat{\theta} = \arg \min_{\theta_h} \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta_b, \theta_h}(x_i), y_i) \quad \text{con } \theta_b = \theta_b^{(0)} \text{ fijo o casi fijo.} \quad (4)$$

Tres regímenes:

- **Feature extraction.** Backbone congelado (θ_b fijo). Sólo cabeza entrenada. Coste mínimo, datos mínimos.

- **Fine-tuning ligero.** Última o últimas dos capas del backbone descongeladas, con η pequeño (típicamente $\eta_b = \eta_h/10$).
- **Fine-tuning completo.** Todo el backbone descongelado. Se usa si hay $\geq 10^4$ ejemplos en $\mathcal{D}_{\text{tgt}}$.

IDEA CLAVE. Con transformers de Hugging Face, una receta de transfer ocupa ~ 15 líneas: cargar `AutoModelForImageClassification`, sustituir la cabeza con `num_labels` del dominio y entrenar con `Trainer`.

6. Concepto $\rightarrow$ aplicación de negocio

- **Control de calidad visual** (Iberdrola, Repsol): backbone ResNet-50 + cabeza binaria, fine-tune ligero.
- **Retail visual** (Mercadona, Zara): detector YOLO/DETR sobre catálogo + clasificador de prenda.
- **OCR de facturas** (BBVA): DiT (Document Image Transformer) fine-tuneado, ~ 1500 facturas etiquetadas.

La métrica clave en producción no es la accuracy del benchmark; es el coste operacional de un falso positivo o falso negativo. Optimizar (4) con la pérdida ponderada por coste: $\ell_{\text{cost}}(\hat{y}, y) = c_{\text{FN}} y(1 - \hat{y}) + c_{\text{FP}}(1 - y)\hat{y}$.